

Alzikrayat — Architecture Report

رحيق عبد العظيم محمد الحسن احمد

قسم تقانة المعلومات

رابط: [raheeg007-creator/project](https://github.com/raheeg007-creator/project)

1. Project Overview

Alzikrayat is a photo-sharing web application that allows registered users to upload photos, browse a shared gallery, and comment on each other's uploads. The project was built entirely in raw PHP, without any backend framework (Laravel, Symfony) or ORM (Eloquent, Hibernate), as required by the course specification. The goal was to implement the underlying mechanics of a web application — routing, session handling, database access, and authentication — by hand.

2. Architectural Approach

The system follows the Model-View-Controller (MVC) pattern layered on top of a physical 3-tier structure, as specified in the project guide. The two structures map onto each other as follows:

3-Tier Layer	MVC Component	Implementation
Presentation	Views	PHP templates in views/, styled with Bootstrap
Application / Business	Controllers	AuthController, PhotoController, CommentController
Data	Models	User, Photo, Comment (raw PDO queries)

All controllers extend an abstract base Controller class that provides shared helpers (view rendering, redirects, HTTP method checks). Similarly, all models extend a base Model class that supplies a shared PDO connection obtained from a Database singleton, so no model or controller manages its own database connection.

3. Routing Engine

Since no framework routing was allowed, a manual Router class was written from scratch. Routes are registered with an HTTP method, a path pattern, and a [Controller, action] pair. At request time, the Router converts each route's {param} placeholders into a regular expression capturing group, then matches the current request path against every registered pattern until one succeeds:

`'/photo/{id}' → '#^/photo/([^\s/]+)$#'`

Captured values (such as the numeric id in /photo/25) are extracted from the regex match and passed directly as arguments to the matched controller action. If no route matches, the Router serves a styled 404 page.

4. Database Design

The schema consists of three normalized tables: users, photos, and comments. Both photos and comments hold a foreign key back to users, and comments additionally reference photos — both with ON DELETE CASCADE, so deleting a user or a photo automatically removes its dependent records. All primary keys are auto-incrementing integers.

Every query in the Model layer is written by hand using PDO prepared statements with named placeholders (e.g. :email), which is the primary defense against SQL injection.

5. Authentication and Security

- Passwords are hashed with PHP's `password_hash()` (Bcrypt) before storage; raw passwords are never saved.
- Login state is tracked with PHP sessions (`$_SESSION`), destroyed on logout.
- A separate 'last login' cookie (independent of the session) is set on every successful login and expires after 7 days, satisfying the persistent browser-state requirement.
- Every dynamic output in the views is passed through `htmlspecialchars()` to prevent XSS.
- Photo deletion checks ownership at the query level (`WHERE id = :id AND user_id = :userId`), so a user can never delete another user's photo even if they forge the request.
- File uploads are validated on both the client (JavaScript) and server (PHP `mime_content_type()` and file size checks) before being moved into `public/images/uploads/` with a generated unique filename.

6. Input Validation

All forms enforce validation on three layers, as required by the specification: HTML5 attributes (`required`, `pattern`, `maxlength`) provide the first check in the browser; a JavaScript listener on form submission provides a second, more descriptive check; and the corresponding Controller method repeats the validation server-side, since client-side checks can always be bypassed. Server-side validation is treated as the only authoritative layer.

7. Challenges and Decisions

- Base path handling: because the app is served from `/project/public/` rather than the domain root, a `BASE_PATH` constant was added to the base Controller so that every `redirect()` call resolves correctly without repeating the path in every controller.

- Regex anchoring in the Router: an early version of the Router matched routes as substrings (e.g. '/photo' matching '/photo/25' too). Adding ^ and \$ anchors around each compiled pattern fixed this.
- File upload trust: the initial file-type check relied on the browser-supplied MIME type, which can be spoofed. It was replaced with PHP's `mime_content_type()`, which inspects the actual file contents on the server.

8. Technology Stack

Backend: raw PHP with PDO. Database: MySQL. Frontend: HTML5, Bootstrap 5 (RTL build), and vanilla JavaScript for client-side validation. Development environment: XAMPP (Apache + MySQL) with VS Code, version-controlled with Git and hosted on GitHub.